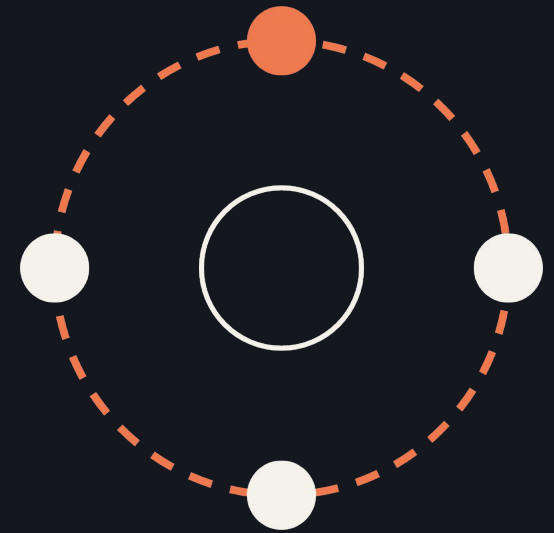


Dundee Data Meetup · 29 September 2026

AI agents and agentic systems

What they are, how they work, and what they mean for us



Steven Wallace · PhD Researcher, University of Aberdeen

ABOUT ME

Steven Wallace: who am I?

From wiring and controlling buildings to researching and teaching AI.

RESEARCH

PhD Researcher, University of Aberdeen

AI, Deep Learning & Remote Sensing

TEACHING

Teaching Assistant

- Data Mining with Deep Learning
- Machine Learning
- Symbolic AI
- Databases with Data Management
- Algorithms and Data Structures
- Distributed Systems

INDUSTRY

- SJIB graded electrician
- Maintenance technician, CNC machinery
- BMS controls engineer
- BMS and energy manager

A building management system is an agent too: it senses, decides and acts to meet a goal.

FOLLOW ALONG

Example code on GitHub

All three travel-booking coding examples from this talk, ready to run.



REPOSITORY

`https://github.com/Stevieeee83/ai-agent-engineering-portfolio`

Coding example 1 Rule-based agent

Coding example 2 Utility functions, then an LLM agent with tools

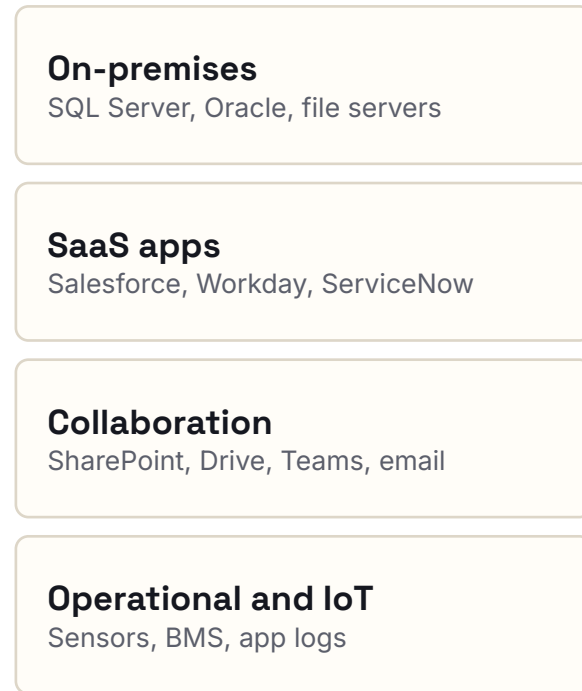
Coding example 3 The full agent: RAG, MCP, Skills and a context graph

THE DATA LANDSCAPE

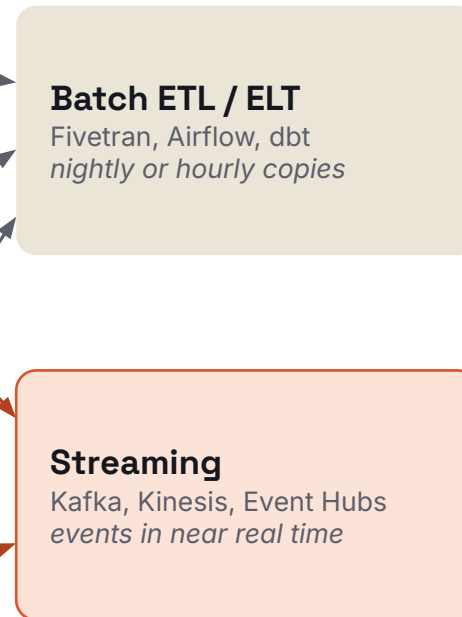
Enterprise data storage

A typical company's data is spread across dozens of systems, clouds and pipelines.

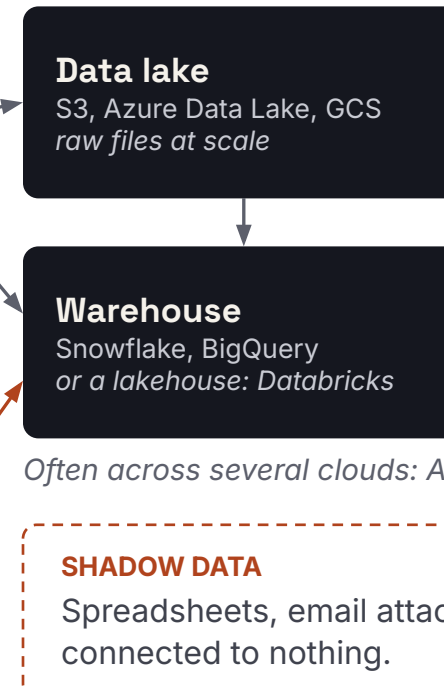
SOURCES



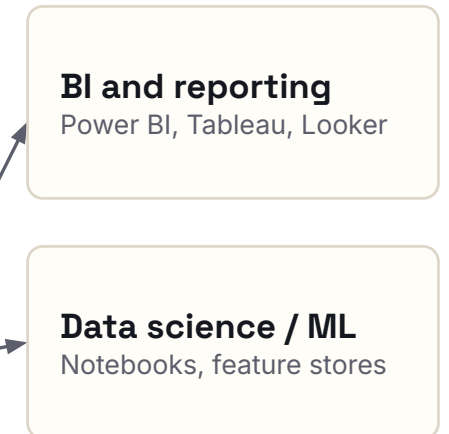
PIPELINES



CLOUD STORAGE



CONSUMERS



No single place holds the full picture, and every copy can drift out of date.

TODAY

Four questions we'll answer

01

**Where did agents
come from?**

Software and symbolic
agents, then LLMs

02

**How does an AI
agent work?**

The loop, tools and levels
of autonomy

03

**What powers
modern agents?**

RAG, MCP, Skills and
context graphs

04

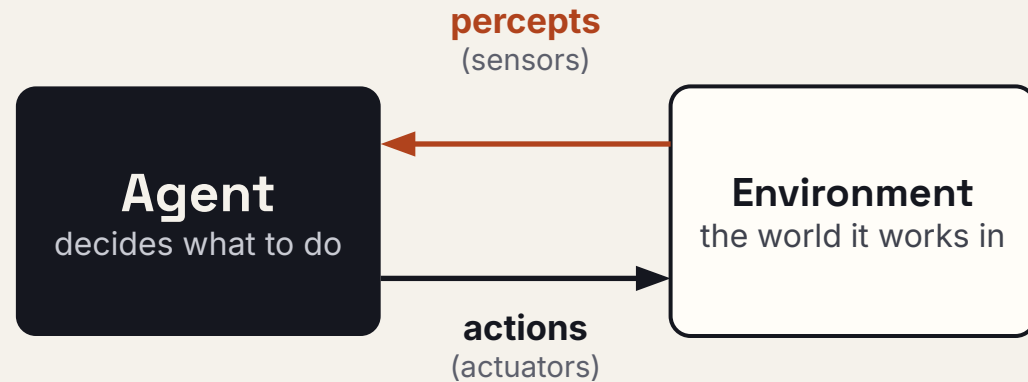
**How do we build
them well?**

Evaluation, limits and first
steps

BACKGROUND

Software agents: what are they?

Software that perceives its environment and acts on it by itself, in pursuit of a goal.



Everyday examples: thermostats, spam filters, web crawlers, game characters and trading bots.

After Russell & Norvig, Artificial Intelligence: A Modern Approach

WOOLDRIDGE & JENNINGS (1995): FOUR PROPERTIES

Autonomy

Operates without a human directing every step.

Reactivity

Notices changes in its environment and responds.

Proactiveness

Takes the initiative to pursue its goals.

Social ability

Communicates with other agents and people.

BACKGROUND

Symbolic AI agents: what are they?

Agents that reason over explicit facts, goals and hand-written rules.

THE CLASSIC BDI ARCHITECTURE

Beliefs

What it holds true about the world.

e.g. Flight BA123 has 4 seats left

Desires

The goals it would like to achieve.

e.g. Get the traveller to Berlin

Intentions

The plan it has committed to.

e.g. Book BA123, then the hotel

Strengths

- Transparent: every decision traces back to a rule
- Predictable, verifiable and easy to test
- Proven in medicine, industry and aerospace

Limits

- Brittle outside the rules it was given
- Knowledge must be hand-coded and maintained
- Struggles with language, ambiguity and common sense

BACKGROUND

Coding example 1: rule-based agent

rule_based_agent.py

```
def book_trip(req):  
    # Beliefs: facts from the flights API  
    flights = get_flights(req.origin, req.dest, req.date)  
  
    # Rules: hand-written by a developer  
    options = [f for f in flights  
               if f.price <= req.budget and f.seats > 0]  
    if not options:  
        return "No flight matches the rules"  
  
    # Intention: commit to the cheapest option  
    return book(min(options, key=lambda f: f.price))
```

WORKS WELL

Deterministic and explainable

Fast, cheap, easy to test, and every booking traces to a rule.

BREAKS WHEN A USER ASKS

“Get me somewhere warm over half-term for under £300.”

No rule knows what “warm” or “half-term” means, and there is no destination to search.

(agent) **steven@steven-Inspiron-5570**:~/Documents/Conferences/Dundee-Data-Meetup/AI-Agents-and-Agent-AI-Workflows-Dundee-Data-Meetup-29-Sep-2026-dev/example_1\$

BACKGROUND

Utility functions

Goals say whether an outcome is acceptable. A utility function says how good it is.

RUSSELL & NORVIG: AGENT TYPES

Simple reflex If this, then that

Model-based Tracks state it can't see

Goal-based Plans to reach a goal

Utility-based Picks the best outcome

Learning Improves from experience

SCORE EACH OUTCOME

$$U = -w_1 \cdot \text{price} - w_2 \cdot \text{hours} - w_3 \cdot \text{stops}$$

CHOOSE THE BEST ACTION

$$a^* = \operatorname{argmax}_a E[U(\text{result}(a))]$$

Trade-offs

Cheaper but slower? The weights decide.

Uncertainty

Expected utility handles delays and risk.

BACKGROUND

Coding example 2 Part 1: utility functions

utility_agent.py

```
WEIGHTS = {"price": 0.5, "hours": 0.3, "stops": 0.2}

def utility(f, req):
    # Normalise each factor to 0..1, higher is better
    price = 1 - f.price / req.budget
    hours = 1 - f.hours / 24
    stops = 1 - f.stops / 3
    return (WEIGHTS["price"] * price
            + WEIGHTS["hours"] * hours
            + WEIGHTS["stops"] * stops)

def book_trip(req):
    flights = get_flights(req.origin, req.dest, req.date)
    options = [f for f in flights if f.price <= req.budget]
    return book(max(options, key=lambda f: utility(f, req)))
```

BETTER THAN RULES

Balances competing goals

Weights price, journey time and stops instead of only taking the cheapest.

STILL BREAKS

Someone has to choose the weights, and every factor must be a number.

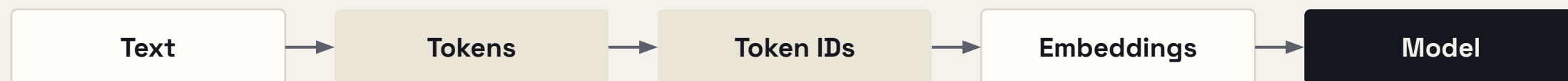
"Somewhere warm over half-term" still can't be scored. Next: let an LLM interpret the request.

(agent) **steven@steven-Inspiron-5570**:~/Documents/Conferences/Dundee-Data-Meetup/AI-Agents-and-Agent-AI-Workflows-Dundee-Data-Meetup-29-Sep-2026-dev/example_2\$

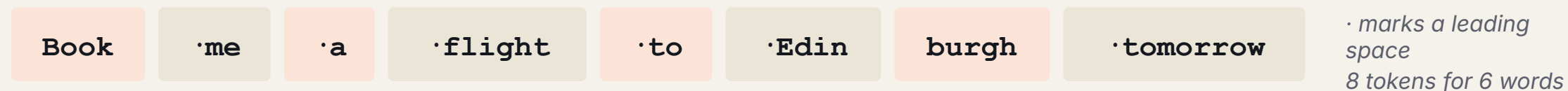
BACKGROUND

Tokenization

LLMs don't read words. They read tokens: sub-word pieces mapped to integer IDs.



TOKENS



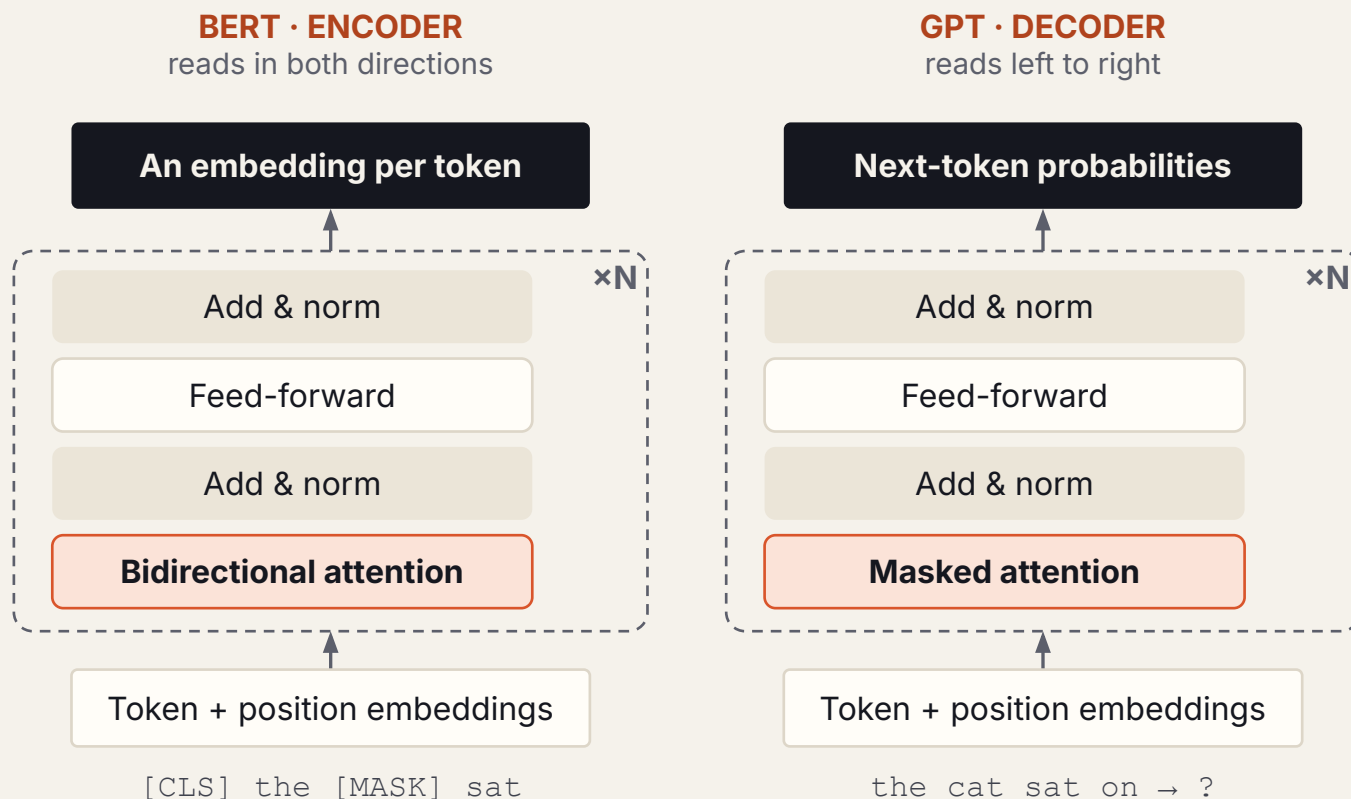
TOKEN IDS (ILLUSTRATIVE)

7280	757	264	11213	311	3199	12799	16986
Sub-word pieces Byte-pair encoding learns common chunks: frequent words are one token, rare ones split.	Rule of thumb In English, one token is roughly four characters, or about three-quarters of a word.			Why agents care Context windows, prices and rate limits are all counted in tokens, including every tool result.			

BACKGROUND

Large language models (LLMs)

Transformer networks trained on vast amounts of text to predict missing or next tokens.



Attention

Each token weighs the other tokens it can see, building up context.

Encoders (BERT)

See the whole input at once. Strong at understanding: classification, search, embeddings.

Decoders (GPT)

See only earlier tokens and predict the next. Strong at generation: the basis of chat models.

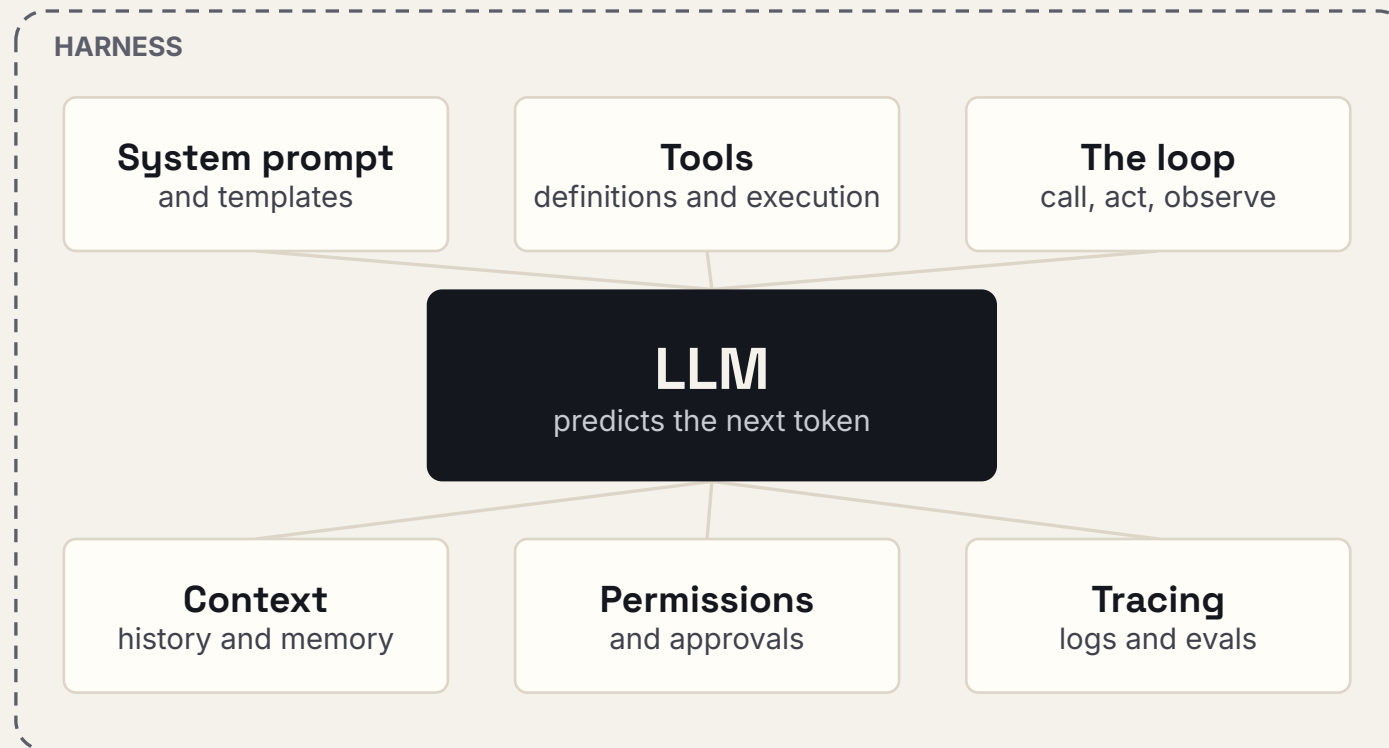
Why agents care

A decoder that follows instructions can also write a plan or a tool call.

BACKGROUND

The LLM harness

The software wrapped around a model that turns it into something useful.



THE ANALOGY

The model is the engine. The harness is the rest of the car.

You already use harnesses

ChatGPT, Claude Code and Cursor are harnesses. Same model, different harness, very different results.

BACKGROUND

LLM prompt templates

Reusable prompts with slots your code fills at run time: the interface between software and the model.

travel_prompt.txt

SYSTEM

You are a travel booking assistant.
Only recommend flights listed in CONTEXT.

CONTEXT

{flight_results}

TASK

Find the best option for: **{user_request}**
Budget: **{budget}**

OUTPUT

JSON with keys flight_id and reason

ROLE

Who the model should be, and its ground rules.

CONTEXT

Data your code injects: search results, documents, history.

TASK

The variable part: the user's request and its parameters.

OUTPUT FORMAT

Something your code can parse, such as JSON.

THE ONE-SENTENCE VERSION

An AI agent is a model that pursues a goal: it plans, uses tools, checks its work, and keeps going until the job is done.

WHAT IS AN AGENT?

Four ingredients make an agent



Model

The reasoning engine: a large language model that reads, plans and decides.



Tools

Ways to act: search the web, run code, send an email, update a spreadsheet.



Memory

What it carries between steps: notes, files and earlier results.



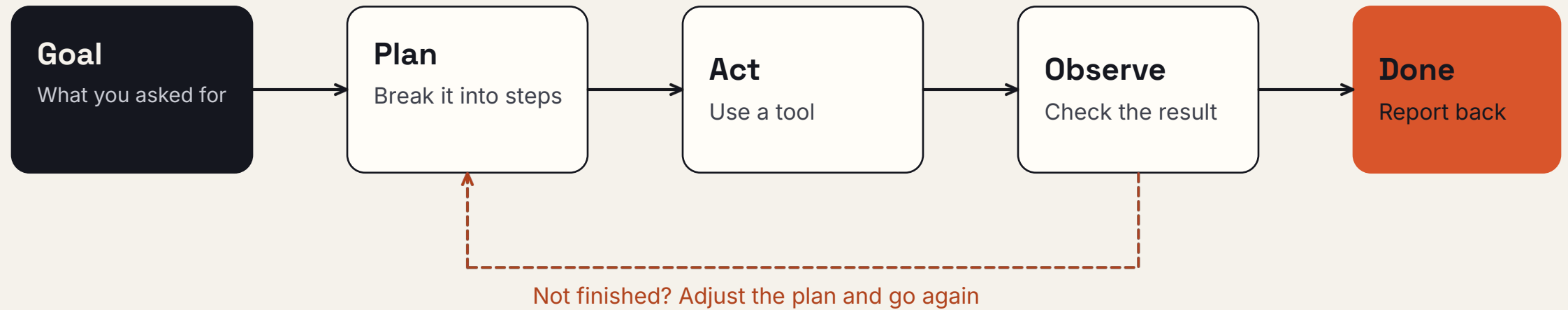
Loop

Repeat until done: act, look at what happened, adjust.

HOW DO THEY WORK?

The agent loop

Agents repeat this cycle, sometimes dozens of times, for one request.



HOW DO THEY WORK?

Coding example 2 Part 2: an LLM agent

llm_agent.py

```
tools = [search_flights, get_weather,
         check_calendar, book_flight]
messages = [system_prompt, user_request]

while True:
    reply = llm(messages, tools=tools)
    if not reply.tool_calls:
        return reply.text          # done
    for call in reply.tool_calls:
        result = run_tool(call)    # act
        messages += [reply, result] # observe
```

SAME REQUEST, NEW RESULT

“Somewhere warm over half-term for under £300.”

- **check_calendar** → finds half-term dates
- **get_weather** → picks warm places
- **search_flights** → filters under £300
- Asks you before **book_flight**

WHAT CHANGED

The rules moved into the model. Your code now owns the tools, limits and guardrails.

(agent) **steven@steven-Inspiron-5570**:~/Documents/Conferences/Dundee-Data-Meetup/AI-Agents-and-Agent-AI-Workflows-Dundee-Data-Meetup-29-Sep-2026-dev/example_2\$

HOW DO THEY WORK?

A spectrum of autonomy

Less autonomy · you steer

More autonomy · it steers

Chat assistant

You ask, it answers. You do the work.

Copilot

Suggests next steps inside your tools. You approve each one.

Workflow agent

Runs a set multi-step process, such as sorting an inbox.

Autonomous agent

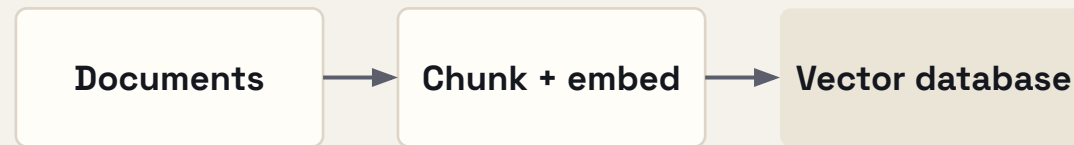
Given a goal, chooses its own steps and tools.

THE MODERN AGENT TOOLKIT

Retrieval augmented generation (RAG)

Give the model the right documents at question time, instead of retraining it.

INDEX · OFFLINE, AS DOCUMENTS CHANGE



QUERY · ONLINE, EVERY REQUEST



Why it helps

Fresh, private or specialist knowledge, and answers that can cite their sources.

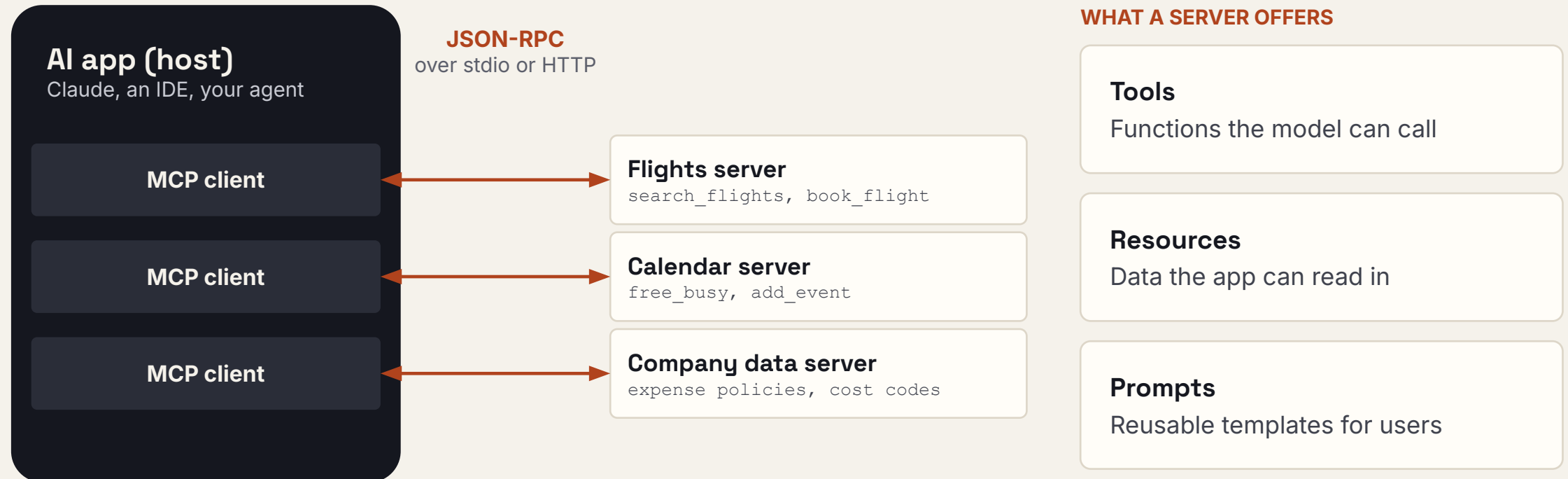
In the travel agent

Travel policies, visa rules and hotel guides, retrieved for each request.

THE MODERN AGENT TOOLKIT

Model Context Protocol (MCP)

An open standard for connecting AI applications to tools and data.



Launched by Anthropic in November 2024; donated to the Linux Foundation's Agentic AI Foundation in December 2025.

Agent Skills

Folders of instructions, scripts and references that an agent loads only when a task needs them.

skills/book-travel

```
book-travel/  
├── SKILL.md  
├── scripts/compare_fares.py  
└── references/travel_policy.md  
  
---  
name: book-travel  
description: Book business travel within  
             company policy. Use when asked to plan  
             or book a trip.  
---  
1. Read references/travel_policy.md  
2. Run scripts/compare_fares.py  
3. Ask before booking over the limit
```

PROGRESSIVE DISCLOSURE

1 • Metadata

Name and description, always in context
(about 100 tokens)

2 • Instructions

The SKILL.md body, loaded when the task matches

3 • Resources

Scripts and references, loaded only if needed

MCP gives an agent access. Skills give it know-how.

Open standard (agentskills.io), released by Anthropic in December 2025

THE MODERN AGENT TOOLKIT

Context stuffing: the actual prompt

Everything from the toolkit ends up as tokens in one prompt, rebuilt and sent again on every turn.

the actual prompt, abridged

```
[system]    You are a travel booking assistant...
[tools]     search_flights(origin, dest, date)...
[skills]    book-travel: Book business travel...
[retrieved] travel_policy.md: max fare £300 unless
[precedent] trip 4821: £340 approved, conf. rate
[history]   user: Book me to Berlin in March
            tool: 3 flights found (1,240 tokens)
[user]      Book the 08:40 one, please.
```

Why stuffing hurts

Every token is paid for on every turn, and key facts get lost in a crowded prompt.

What good harnesses do

Retrieve less but better, summarise history, load skills on demand.

AN ILLUSTRATIVE CONTEXT WINDOW



Skills, Precedent and User are the narrow segments between Tools and History.

Context graphs

A connected record of decision traces, so an agent can look up why things were done before.



Knowledge graph

Stores what is true: entities and the relationships between them.

Context graph

Adds why: the inputs, policies, exceptions, approvals and outcomes behind each decision.

Next time a fare breaks policy, the agent can find this precedent and who approved it.

An emerging idea, popularised from late 2025; the terminology is still settling.

THE MODERN AGENT TOOLKIT

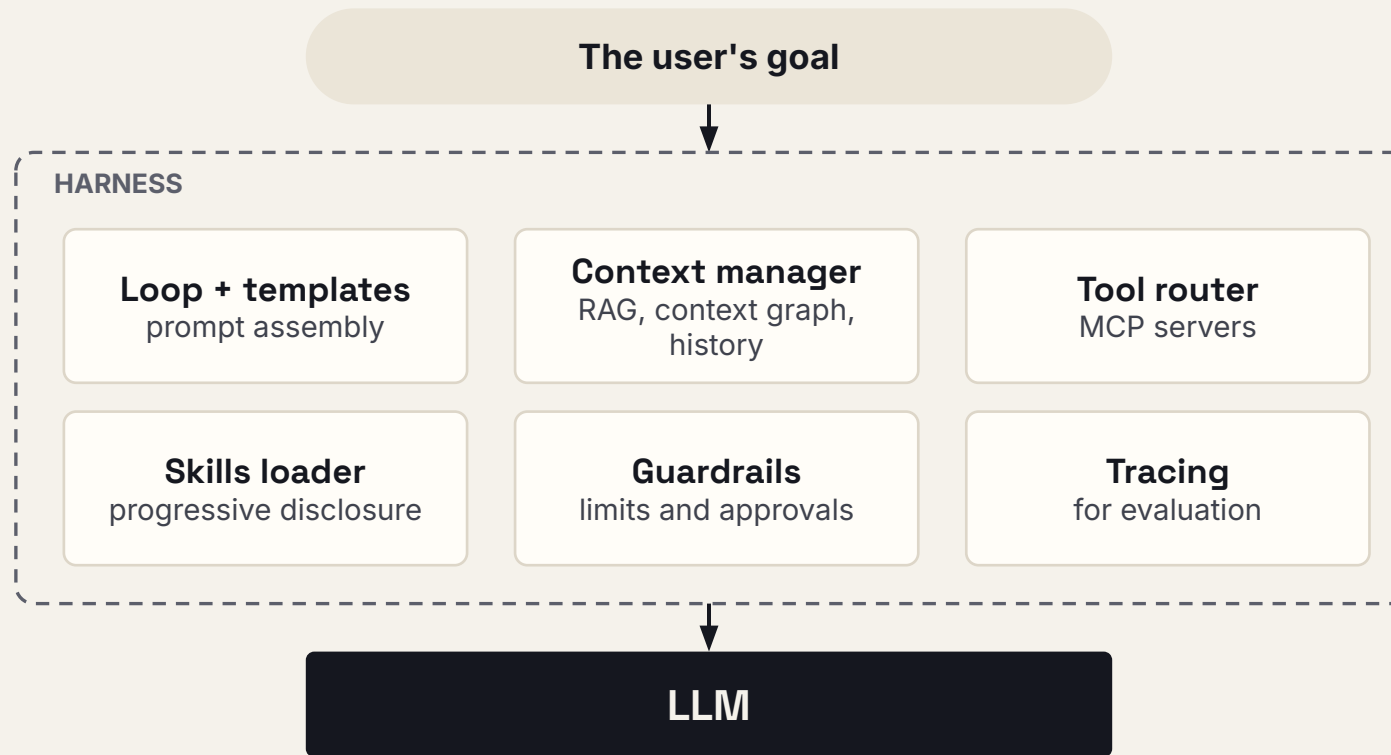
Putting it all together

LLMs with RAG, MCP, Skills and context graphs: each answers a different question for the agent.



The LLM harness, revisited

The toolkit plugs into the harness, and the harness decides what reaches the model.



Build or buy?

Agent frameworks and SDKs give you a harness. Coding example 3 shows the parts you would otherwise build.

Where quality comes from

Many agent failures are harness problems: the wrong context, vague tool descriptions, or no limits.

Coding example 3: the full agent

LLM + RAG + MCP + Skills + context graph (illustrative pseudo-code)

full_agent.py

```
agent = Agent(  
    model=llm,  
    system=load_template("travel_agent.txt"),  
    tools=mcp_tools(["flights", "calendar"]),  
    skills=["skills/book-travel"],  
    retriever=vector_store("travel_policies"),  
    memory=context_graph("bookings"),  
    approve=ask_human, # gate risky actions  
)  
  
agent.run("Book me to the Berlin data "  
         "conference in March")
```

WHAT HAPPENS ON ONE REQUEST

- 1 Skills**
Loads the book-travel procedure
- 2 RAG**
Retrieves the conference travel policy
- 3 Context graph**
Finds a similar exception that was approved
- 4 MCP**
Searches flights and checks the calendar
- 5 Human**
Approves the fare; a new decision trace is saved

Context-Engineered Travel Agent

Example 3 · RAG + Context Graph + Skills + MCP · Dundee Data Meetup

Offline planner (set OPENAI_API_KEY for the LLM)

5 MCP tools

30 chunks

58 nodes / 140 edges

New conversation

Who is travelling?

Alex Morgan
Senior data consultant at Acme Analytics

Sam Rivera
Parent planning family holidays · party of 4

Guest (no profile)
Unknown traveller - nothing in the graph

Context augmentation

applies to next message

☒ **RAG**
Retrieve policies & guides from the knowledge base

☒ **Context graph**
Traveller profile, relationships, memory

☒ **Skills**
Load expert procedures on demand

☒ **MCP tools**
Always on - the agent's hands (search & book)

Trip request

From	To
My home (let th	Amsterdam (AM
Depart	Return (optional)
10 / 20 / 2026	mm / dd / yyyy
Travellers	Cabin
1	Economy



Where would you like to go?

Use the trip form, type a request, or try an example:

Get me to Amsterdam for the data summit, with a hotel

I need to be in London on 2026-10-20 for a client meeting

Plan a week in New York in November, what do I need before I go?

Trace RAG Graph Skills Bookings

Send a message to see each step the agent takes.

Ask the travel agent... (e.g. "yes, book it")

Send

Context-Engineered Travel Agent

Example 3 · RAG + Context Graph + Skills + MCP · Dundee Data Meetup

gpt-5-mini

5 MCP tools

30 chunks

58 nodes / 140 edges

New conversation

Who is travelling?

Alex Morgan

Senior data consultant at Acme Analytics

Sam Rivera

Parent planning family holidays · party of 4

Guest (no profile)

Unknown traveller - nothing in the graph

Context augmentation

applies to next message



RAG

Retrieve policies & guides from the knowledge base



Context graph

Traveller profile, relationships, memory



Skills

Load expert procedures on demand



MCP tools

Always on - the agent's hands (search & book)

Trip request

From

My home (let this be)

To

Amsterdam (AM)

Depart

10 / 20 / 2026



Return (optional)

mm / dd / yyyy



Travellers

1



Cabin

Economy



Where would you like to go?

Use the trip form, type a request, or try an example:

Get me to Amsterdam for the data summit, with a hotel

I need to be in London on 2026-10-20 for a client meeting

Plan a week in New York in November, what do I need before I go?

Trace

RAG

Graph

Skills

Bookings

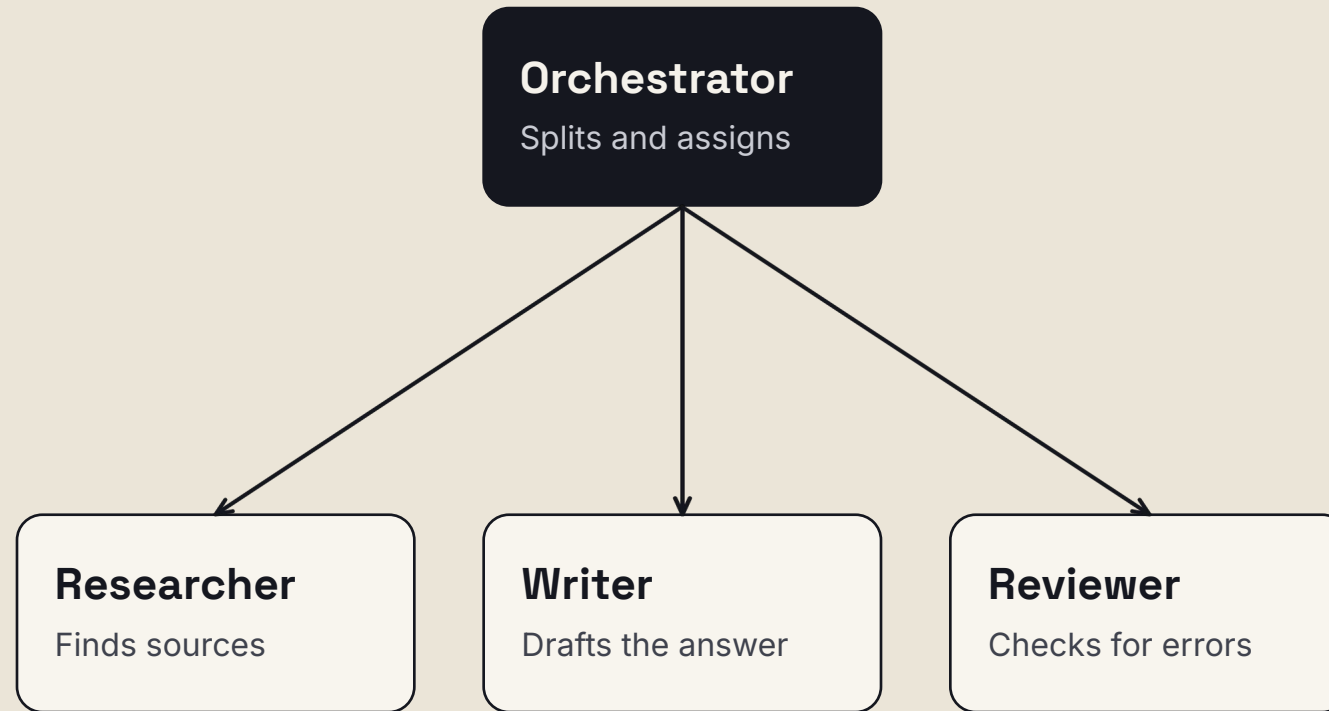
Send a message to see each step the agent takes.

Ask the travel agent... (e.g. "yes, book it")

Send

HOW DO THEY WORK?

Agents can work in teams



Divide

An orchestrator splits a big job into pieces.

Specialise

Each specialist has its own instructions and tools.

Combine

Results come back, get checked, and are merged.

HOW DO THEY WORK?

Where you'll meet agents today



Coding

Agents that read a codebase, make changes and run the tests.



Research

Assistants that search many sources and write a referenced summary.



Customer support

Agents that look up an order, process a refund, and hand off to a person.



Browsing

Agents that fill in forms, compare prices or book appointments.

WHY NOW?

Why agents arrived now

1 Better reasoning

Models got much better at planning and following multi-step instructions.

2 Standard tool use

Shared protocols such as MCP let models connect to apps and data.

3 Longer memory

Models can hold whole documents, codebases and long conversations.

4 Cheaper, faster runs

Many-step loops became affordable enough to be practical.

WHERE DO THEY STRUGGLE?

If each step goes right 90% of the time, a ten-step task succeeds only about a third of the time.

$0.9 \times 0.9 \times \dots$ ten times ≈ 0.35

WHERE DO THEY STRUGGLE?

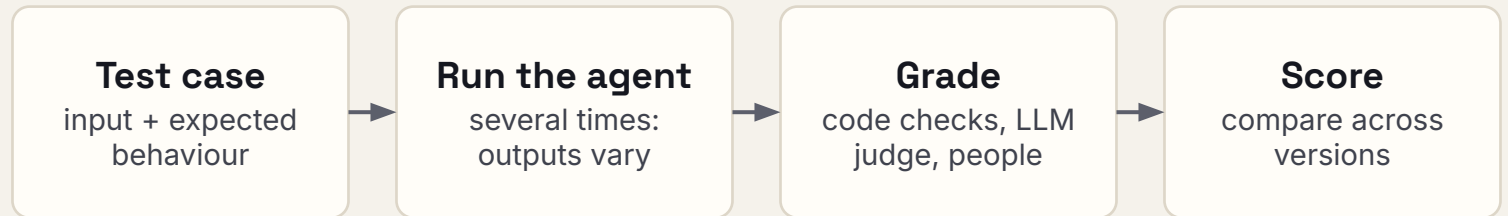
Evaluating an agentic AI system

50

test cases

Each as close as possible to the real human behaviour the system is trying to replicate.

e.g. how a travel coordinator actually handles a request



WHAT TO MEASURE

Outcome

Did it complete the task correctly?

Trajectory

Right tools, right order, no wasted steps?

Safety

Stayed within policy and permissions?

Cost and latency

Tokens, model calls and time per task

WHERE DO THEY STRUGGLE?

Honest limits to know about

Limit	What it looks like
Confident errors	Invents a fact or misreads a result, then builds on it
Prompt injection	Hidden instructions in a web page or email hijack the agent
Too much access	An agent with broad permissions can do broad damage
Cost and speed	Many loops mean many model calls, which take time and money
Accountability	When an agent acts, who answers for the outcome?

WHAT CAN WE DO?

Principles for using agents well

Keep a human in the loop

Approve anything irreversible:
payments, emails, deletions.

Give least access

Only the accounts and permissions the
task actually needs.

Start small

One well-defined task before a whole
workflow.

Watch what it does

Keep logs and review its steps, not just
its answer.

Test before trusting

Try it on cases where you already know
the right answer.

Be transparent

Tell people when they're dealing with an
agent.

WHAT CAN WE DO?

What this could mean for us



Data teams

Agents that profile a new dataset, draft SQL and write the data-quality report for review.



Software teams

Coding agents that triage issues, write tests and open pull requests for a person to review.



This meetup

Build a skill or MCP server for a tool you use, and share it at a future session.

WHAT CAN WE DO?

Getting started this week

1 Rebuild coding example 2 with your own model API key and a single tool.

2 Wrap one data source you use as an MCP server.

3 Write a SKILL.md for a task your team repeats.

4 Write your test cases before giving it more autonomy.

WHAT CAN WE DO?

Agentic AI system frameworks

Framework	From	Good for
LangGraph	LangChain	Stateful graphs with fine control over the flow
CrewAI	CrewAI	Role-based teams of collaborating agents
OpenAI Agents SDK	OpenAI	Lightweight agents with handoffs and guardrails
Claude Agent SDK	Anthropic	The Claude Code harness: tools, skills and MCP built in
Agent Development Kit	Google	Multi-agent apps, especially on Gemini and Vertex AI
Microsoft Agent Framework	Microsoft	Successor to AutoGen and Semantic Kernel
Pydantic AI	Pydantic	Type-safe Python agents with structured outputs

You don't need a framework to start: coding example 2 is a complete agent. The landscape moves quickly, so check versions before the talk.

Thank you

Questions?



Email



LinkedIn



X (Twitter)